Q1. What is Machine Learning? Explain the main types with examples.

Machine Learning (ML) is a branch of Artificial Intelligence in which computers learn patterns from data and improve their performance on a task without being explicitly programmed with rules. Instead of writing fixed instructions, we give an algorithm examples, and it builds a model that can make predictions or decisions on new data.

Main types of Machine Learning:

Type	How it works	Examples
Supervised Learning	Learns from labelled data (inputs with known outputs) to predict the output for new inputs.	House price prediction, spam email detection, disease diagnosis
Unsupervised Learning	Finds hidden structure in unlabelled data; no target variable.	Customer segmentation (K-Means), anomaly detection, PCA
Semi-Supervised Learning	Uses a small amount of labelled data with a large amount of unlabelled data.	Image/text classification with few labels
Reinforcement Learning	An agent learns by interacting with an environment and receiving rewards or penalties.	Game playing (chess, Go), robotics, self-driving cars

Q2. What is Supervised Learning? Difference between Regression and Classification.

Supervised Learning is a type of ML where the model is trained on a dataset containing both input features (X) and the correct output label (y). The model learns the mapping from X to y and then predicts y for unseen data. Its two main problem types are Regression and Classification.

Basis	Regression	Classification
Output	Continuous numerical value	Discrete category / class label
Goal	Predict a quantity	Assign data to a class
Examples	House price, temperature, salary	Spam / not spam, disease yes / no, digit recognition
Algorithms	Linear Regression, Ridge, Lasso, SVR	Logistic Regression, Decision Tree, KNN, SVM
Metrics	MAE, MSE, RMSE, R^2	Accuracy, Precision, Recall, F1, ROC-AUC

Q3. What is Linear Regression? Equation, intercept and coefficient.

Linear Regression is a supervised learning algorithm that models the relationship between a continuous dependent variable (y) and one or more independent variables (X) by fitting a straight line (or hyperplane) that minimises the sum of squared errors between actual and predicted values (Ordinary Least Squares).

Simple Linear Regression:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

Multiple Linear Regression:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n + \varepsilon$$

- **Intercept (β_0):** the predicted value of y when all independent variables are zero. It is the point where the regression line crosses the y-axis, i.e. the baseline value.
- **Coefficient (β_i):** the change in y for a one-unit increase in x_i, keeping all other variables constant. Its sign shows direction (positive = y rises, negative = y falls) and its magnitude shows strength.
- **Error term (ε):** the part of y that the model cannot explain (random noise).

Q4. Evaluation metrics used in Linear Regression

Metric	Formula	Explanation
SSE (Sum of Squared Errors)	$\Sigma (y_i - \hat{y}_i)^2$	Total of squared differences between actual and predicted values. Lower is better; it grows with data size, and it is the quantity OLS minimises.
MSE (Mean Squared Error)	$(1/n)\ \Sigma (y_i - \hat{y}_i)^2$	Average squared error (SSE ÷ n). Penalises large errors heavily; units are squared.
RMSE (Root Mean Squared Error)	$\sqrt{MSE}$	Square root of MSE; in the same unit as the target, so easy to interpret as the typical prediction error.
R^2 Score	$1 - SSE / SST$, where $SST = \Sigma(y_i - \bar{y})^2$	Proportion of variance in y explained by the model. Ranges up to 1; closer to 1 is better. 0 means no better than predicting the mean.

Adjusted R^2	$1 - (1 - R^2)(n - 1)/(n - k - 1)$	R^2 corrected for the number of predictors (k). It increases only if a new variable truly improves the model, so it prevents overfitting by adding useless features.

Q5. Machine Learning workflow: data collection to model evaluation

- **1. Problem definition:** decide what is to be predicted and how success will be measured.
- **2. Data collection:** gather data from databases, files, APIs, sensors or surveys.
- **3. Data cleaning / preprocessing:** handle missing values, duplicates and outliers; fix data types; encode categorical variables; scale features where needed.
- **4. Exploratory Data Analysis (EDA):** summary statistics, distributions, pairplots and correlation heatmaps to understand patterns and relationships.
- **5. Feature selection / engineering:** choose or create the most relevant input variables.
- **6. Train-test split:** divide data (e.g. 80% train / 20% test) so the model is judged on unseen data.
- **7. Model selection and training:** choose an algorithm (e.g. Linear Regression) and fit it on the training set.
- **8. Prediction:** use the trained model to predict on the test set.
- **9. Model evaluation:** compute metrics (MAE, MSE, RMSE, R^2, Adjusted R^2) and check for over/underfitting.
- **10. Tuning, deployment and monitoring:** improve the model, deploy it and monitor its performance over time.

Part B - Practical Questions (Boston Housing)

The complete code is given first, followed by the outputs and answers for each question. The dataset has 506 rows and 14 columns with no missing values. All results below were produced by running this code with random_state = 42.

Complete Python Code

```
# ===== Q6: Load and explore =====
import pandas as pd, numpy as np
import matplotlib.pyplot as plt, seaborn as sns
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

df = pd.read_csv("BostonHousing.csv")
print(df.head())
df.info()
print(df.describe())
print(df.isnull().sum())

# ===== Q7: EDA =====
sns.pairplot(df[["medv","lstat","rm","ptratio","indus","tax","crim"]], corner=True)
plt.show()

plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap="coolwarm", center=0)
plt.title("Correlation Heatmap")
plt.show()
print(df.corr()["medv"].drop("medv").sort_values())

# ===== Q8: Features and target =====
X = df.drop(columns="medv")
y = df["medv"]

# ===== Q9: 80/20 split =====
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)

# ===== Q10: Train, predict, evaluate =====
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

mae  = mean_absolute_error(y_test, y_pred)
mse  = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_test, y_pred)
n, k = X_test.shape
adj_r2 = 1 - (1 - r2) * (n - 1) / (n - k - 1)
print(mae, mse, rmse, r2, adj_r2)

print("Intercept:", model.intercept_)
for name, c in zip(X.columns, model.coef_):
        print(name, c)
```

Q6. Data exploration: head(), info(), describe()

df.head()

```
      crim    zn  indus  chas    nox    rm   age     dis  rad  tax  ptratio       b  lstat  medv
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296     15.3  396.90   4.98  24.0
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242     17.8  396.90   9.14  21.6
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242     17.8  392.83   4.03  34.7
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222     18.7  394.63   2.94  33.4
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222     18.7  396.90   5.33  36.2
```

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   crim     506 non-null    float64
 1   zn       506 non-null    float64
 2   indus    506 non-null    float64
 3   chas     506 non-null    int64  
 4   nox      506 non-null    float64
 5   rm       506 non-null    float64
 6   age      506 non-null    float64
 7   dis      506 non-null    float64
 8   rad      506 non-null    int64  
 9   tax      506 non-null    int64  
 10  ptratio  506 non-null    float64
 11  b        506 non-null    float64
 12  lstat    506 non-null    float64
 13  medv     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

df.describe()

```
        count   mean     std    min     25%     50%    75%     max
crim    506.0   3.614  8.602  0.006   0.082   0.257  3.677  88.976
zn      506.0  11.364  23.322          0.000   0.000  0.000  12.500  100.000
indus   506.0  11.137 6.860  0.460   5.190   9.690  18.100  27.740
chas    506.0   0.069  0.254  0.000   0.000   0.000  0.000   1.000
nox     506.0   0.555  0.116  0.385   0.449   0.538  0.624   0.871
rm      506.0   6.285  0.703  3.561   5.885   6.208  6.624   8.780
```

age	506.0	68.575	28.149	2.900	45.025	77.500	94.075	100.000
dis	506.0	3.795	2.106	1.130	2.100	3.207	5.188	12.126
rad	506.0	9.549	8.707	1.000	4.000	5.000	24.000	24.000
tax	506.0	408.237	168.537	187.000	279.000	330.000	666.000	711.000
ptratio	506.0	18.456	2.165	12.600	17.400	19.050	20.200	22.000
b	506.0	356.674	91.295	0.320	375.378	391.440	396.225	396.900
lstat	506.0	12.653	7.141	1.730	6.950	11.360	16.955	37.970
medv	506.0	22.533	9.197	5.000	17.025	21.200	25.000	50.000

Observations: the dataset has 506 records and 14 numeric columns (11 float, 3 integer) with 0 missing values. The average house price (medv, in $1000s) is about 22.5 with a range of 5 to 50. The target is capped at 50, and crim and zn are highly skewed (mean much larger than median).

Q7. Exploratory Data Analysis

(a) Pairplot

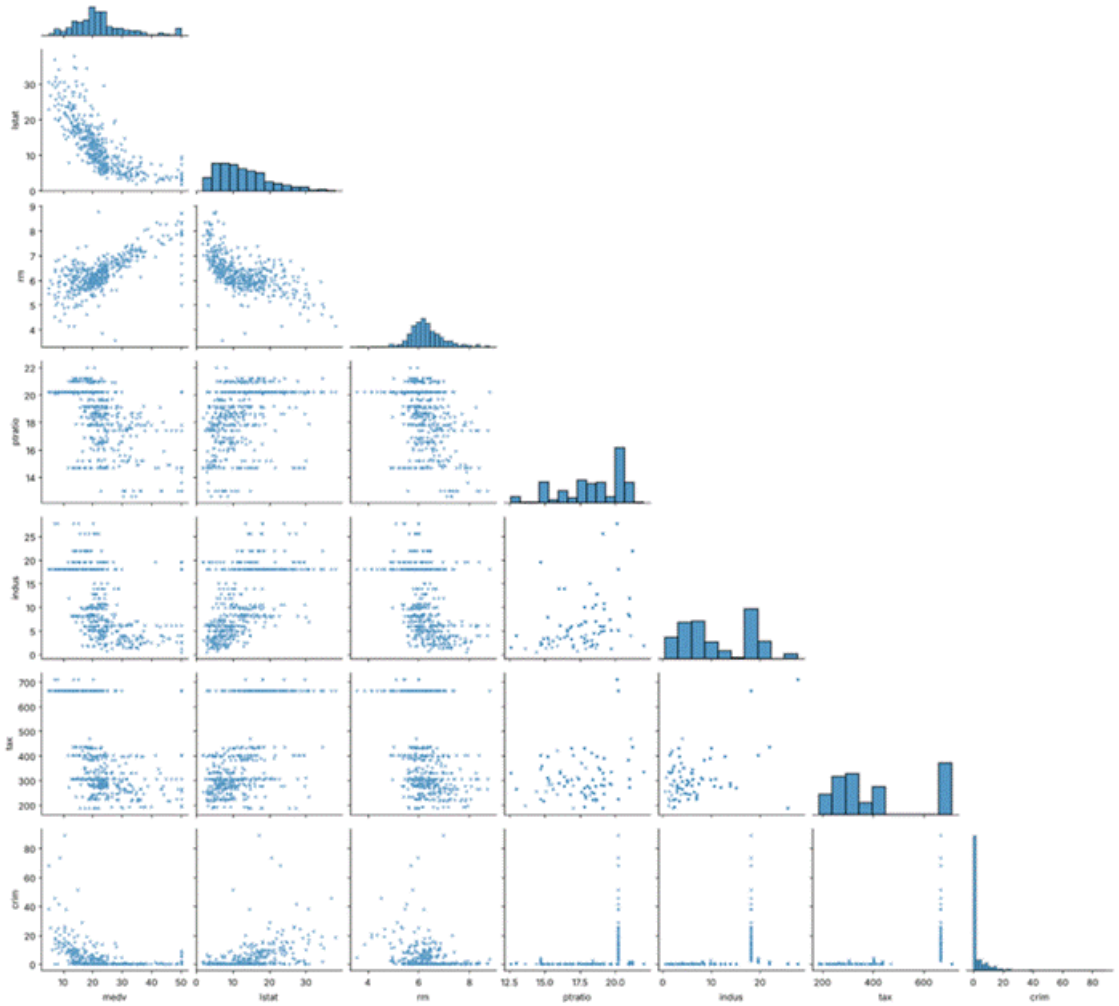


Figure 1: Pairplot of medv with key features (lstat, rm, ptratio, indus, tax, crim)

The pairplot shows a clear upward relationship between rm and medv, and a curved downward relationship between lstat and medv. Crime rate (crim) is heavily right-skewed with most observations near zero.

(b) Correlation Heatmap

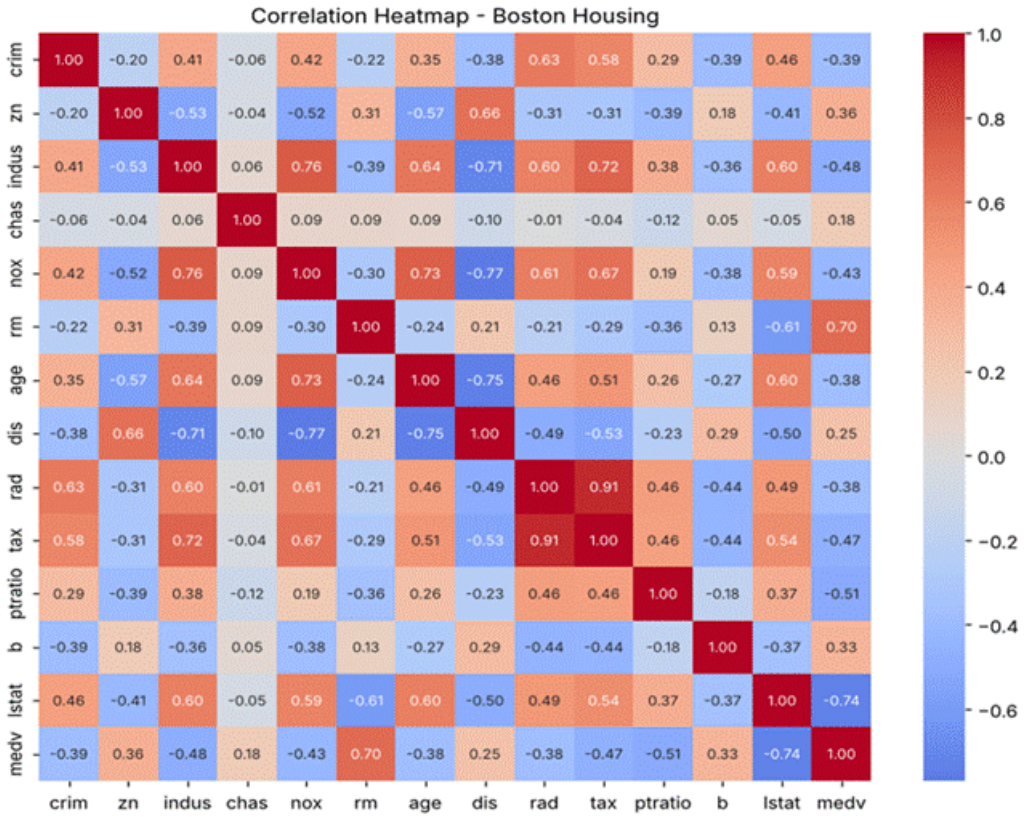


Figure 2: Correlation heatmap of all variables

(c) Features most correlated with house price (medv)

Feature	Correlation with medv	Meaning
lstat	-0.738	Strong negative
rm	0.695	Strong positive
ptratio	-0.508	Strong negative
indus	-0.484	Moderate negative
tax	-0.469	Moderate negative
nox	-0.427	Moderate negative
crim	-0.388	Moderate negative

rad	-0.382	Moderate negative
age	-0.377	Moderate negative
zn	0.360	Moderate positive
b	0.333	Moderate positive
dis	0.250	Weak positive
chas	0.175	Weak positive

Conclusion: lstat (−0.738) and rm (+0.695) are the most highly correlated with house price. More rooms means higher price; a higher share of lower-status population means lower price. ptratio (−0.508), indus (−0.484) and tax (−0.469) also show a notable negative correlation. Note that rad and tax are strongly correlated with each other (about 0.91), which indicates multicollinearity.

Q8. Independent and dependent variables

- **Target (dependent) variable:** medv (median house value in $1000s).
- **Features (independent variables):** the other 13 columns - crim, zn, indus, chas, nox, rm, age, dis, rad, tax, ptratio, b, lstat.

```
X = df.drop(columns="medv")
y = df["medv"]
```

Q9. Train-test split (80% / 20%)

```
X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42)
```

Result: 404 training rows (80%) and 102 testing rows (20%).

Q10. Model training, predictions and evaluation

Sample predictions on the test set

#	**Actual medv**	**Predicted medv**
1	23.6	29
2	32.4	36.03
3	13.6	14.82
4	22.8	25.03
5	16.1	18.77
6	20	23.25
7	17.8	17.66
8	14	14.34

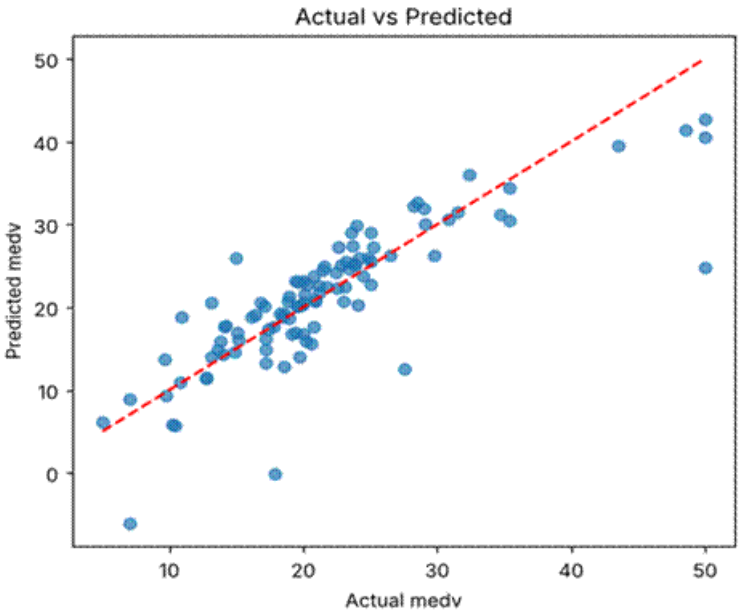


Figure 3: Actual vs predicted prices on the test set (dashed line = perfect prediction)

Model evaluation metrics (test set)

Metric	Value	Interpretation
MAE	3.189	Predictions are off by about $3,189 on average
MSE	24.291	Average squared error
RMSE	4.929	Typical error is about $4,929
R^2 Score	0.6688	The model explains about 66.9% of variance in price
Adjusted R^2	0.6198	R^2 adjusted for 13 predictors (n = 102)

Training R^2 is 0.7509 against a test R^2 of 0.6688, so there is mild overfitting but the model generalises reasonably well. The remaining error suggests some non-linear patterns (e.g. lstat) that a plain linear model cannot capture.

Regression equation

$$\text{medv} = 30.247 - 0.113(\text{crim}) + 0.030(\text{zn}) + 0.040(\text{indus}) + 2.784(\text{chas}) - 17.203(\text{nox}) + 4.439(\text{rm}) - 0.006(\text{age}) - 1.448(\text{dis}) + 0.262(\text{rad}) - 0.011(\text{tax}) - 0.915(\text{ptratio}) + 0.012(\text{b}) - 0.509(\text{lstat})$$

(medv is in $1000s; the equation is built from the training data.)

Model coefficients and intercept

Feature	Coefficient	Interpretation

Intercept	30.247	Baseline predicted price (about $30,247) when every feature equals zero; it has no practical meaning on its own.
crim	-0.1131	Higher crime lowers prices slightly: each +1 in crime rate reduces the price by about $113.
zn	0.0301	More large-lot zoning raises prices very slightly (+$30 per unit).
indus	0.0404	Almost no effect (+$40 per unit); small and not a reliable driver.
chas	2.7844	Houses bordering the Charles River are about $2,784 more expensive, all else equal.
nox	-17.2026	The strongest negative effect: +1 in NOX concentration cuts price by about $17,203 (NOX range is only 0.38 − 0.87, so a 0.1 rise = 1,720).
rm	4.4388	The strongest positive effect: each extra room raises price by about $4,439.
age	-0.0063	Older housing stock has a negligible effect (-$6 per 1% increase).
dis	-1.4479	Farther from employment centres lowers price by about $1,448 per unit of distance.
rad	0.2624	Better highway access is linked to higher prices (+$262 per unit), likely proxying for convenience.
tax	-0.0106	Higher property tax reduces price by about $11 per unit.
ptratio	-0.9155	Each extra student per teacher reduces price by about $915, so school quality matters.
b	0.0124	Tiny positive effect (+$12 per unit).
lstat	-0.5086	Each +1% in lower-status population lowers price by about $509; a major negative driver.

Overall interpretation of coefficients

- **Positive coefficients** (rm, chas, rad, zn, indus, b) increase the predicted price. rm (+4.44) is the most influential positive driver, followed by chas (+2.78).
- **Negative coefficients** (nox, dis, ptratio, lstat, crim, tax, age) decrease the predicted price. nox (−17.20) has the largest coefficient in magnitude, but because NOX has a very small range, scale matters; lstat, dis and ptratio are also important.
- **Caution:** every coefficient is interpreted holding all other features constant. Raw coefficients depend on each feature's units, so they should not be compared directly without scaling, and correlated features such as tax and rad can make individual coefficients less stable.

Final Conclusion

A multiple Linear Regression model on the Boston Housing dataset achieved an R^2 of 0.669, an Adjusted R^2 of 0.620 and an RMSE of 4.93 (about $4,929) on unseen data. House prices rise with the number of rooms and fall with a higher lower-status population, pollution, pupil-teacher ratio, distance from employment centres and crime. Accuracy could be improved with feature scaling, transformation of skewed variables, regularisation (Ridge/Lasso) or non-linear models.